



Technical Documentation

Test Harness – run_verity-tests.sh

Team Oriented Project

Group A

Linux Kernel Extension for DM-Verity Authentication, Verification and Setup

Date: **NOVEMBER 2025**

Cooperation with:

**BMW
GROUP**



ROLLS-ROYCE
MOTOR CARS LTD

[BMW Car IT, Ulm](#)

Eva-Helena Zorman

Tobias Kaufmann

Philipp Graf

The `run_verity_tests.sh` script is the Security Behaviour Automated Test Harness for the dm-verity kernel modules. It is a Bash script designed to manage the entire testing lifecycle, from environment setup and image generation to QEMU execution, log parsing, and final security reporting. Its success hinges on its ability to confirm that malicious or corrupted input always results in a controlled kernel rejection (`REJECTED` or `KERNEL_PANIC`) and never a successful boot (`BOOT_SUCCESS`).

1. System Configuration and Test Matrix Definition

The harness begins by establishing the core execution environment and defining the expected security outcomes for each test case.

Directory Setup and Permissions

The script uses strict directory paths and enforces write permissions on the crucial Binaries folder, where the kernel and all test images reside. It uses `sudo chown` and `chmod` to guarantee the current user can create and delete test images, ensuring the environment is stable before execution.

```
2  set -eo pipefail

79  SCRIPT_DIR="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
80  PROJECT_ROOT="$(cd "$SCRIPT_DIR/.." && pwd)"
81  INTEGRITY_SCRIPT="$SCRIPT_DIR/integrity_tests.sh"
82  QEMU_TESTS_SCRIPT="$PROJECT_ROOT/bootloaders/qemu_tests.sh"
83  BINARIES_DIR="$SCRIPT_DIR/Binaries"
84  LOG_DIR="$SCRIPT_DIR/test_logs"
85  RESULTS_FILE="$LOG_DIR/test_results.txt"
86  TEST_TIMEOUT=15
87  BOOT_TIMEOUT=45

88  # Ensure Binaries directory is owned by the current user and writable
89  if [[ ! -w "$BINARIES_DIR" || "$(stat -c '%U' "$BINARIES_DIR")" != "$USER" ]]; then
90      echo "Fixing ownership and permissions on Binaries directory..."
91      sudo chown -R "$USER":"$USER" "$BINARIES_DIR" || { echo "Failed to set ownership on $BINARIES_DIR"; exit 1; }
92      sudo chmod -R a+rwX "$BINARIES_DIR" || { echo "Failed to set permissions on $BINARIES_DIR"; exit 1; }
93  fi
94
```

The initial `set -eo pipefail` command ensures that the script immediately exits on any command failure (`-e`), treats unset variables as errors (`-u` is assumed by the general design philosophy), and flags pipeline failures (`-o pipefail`), guaranteeing robust execution.

Test Matrix (TEST_MODES)

The heart of the validation logic lies in the **TEST_MODES** associative array. This defines the **Ground Truth**: the required security response for each targeted corruption vector.

```
107 declare -A TEST_MODES=(
108     ["metal"]="PANIC:Corrupted header should be rejected"
109     ["sig1"]="PANIC:Invalid signature should be rejected"
110     ["int_overflow"]="REJECT:Integer overflow should be caught"
111     ["buf_overflow"]="REJECT:Oversized metadata should be rejected"
112     ["trunc_meta"]="REJECT:Out-of-bounds metadata should be rejected"
113     ["bad_offsets"]="REJECT:Invalid offsets should be rejected"
114     ["sanitize"]="REJECT:Garbage input should be rejected"
115     ["best_case"]="BOOT:Clean image should boot successfully"
116 )
117
```

The script processes tests strictly according to the **TEST_ORDER** array to maintain reproducibility. A test passes if the actual log analysis result matches the expected behavior (PANIC, REJECT, or BOOT).

2. Prerequisites and Test Image Management

Before running the emulator, the script ensures all dependencies are present and that the necessary corrupted disk images exist.

Prerequisite Checks

The `check_prerequisites` function is responsible for confirming the presence of external tools and project scripts:

```
173 check_prerequisites() {
174     log "INFO" "Checking prerequisites..."
175
176     local missing=()
177     [[ ! -f "$INTEGRITY_SCRIPT" ]] && missing+=("integrity_tests.sh")
178     [[ ! -f "$QEMU_TESTS_SCRIPT" ]] && missing+=("qemu_tests.sh")
179     command -v qemu-system-x86_64 >/dev/null 2>&1 || missing+=("qemu-system-x86_64")
180     command -v python3 >/dev/null 2>&1 || missing+=("python3")
181
182     if [[ ${#missing[@]} -gt 0 ]]; then
183         log "ERROR" "Missing prerequisites: ${missing[*]}"
184         exit 1
185     fi
186
187     if [[ ! -f "$BINARIES_DIR/rootfs.img" ]]; then
188         log "ERROR" "Original rootfs.img not found in $BINARIES_DIR"
189         exit 1
190     fi
191
192     log "INFO" "All prerequisites satisfied"
193 }
```

If the core scripts (`integrity_tests.sh`, `qemu_tests.sh`) or the emulator are missing, execution halts immediately.

Image Generation

The script uses `integrity_tests.sh` to generate the test images. This is handled by a loop that iterates over the **TEST_ORDER** array, creating only the images that are currently missing.

```
243     local test_image="$BINARIES_DIR/rootfs.${mode}.test.img"
244     if [[ ! -f "$test_image" ]]; then
245         log "INFO" "Creating test image for: $mode"
246         if "$INTEGRITY_SCRIPT" "$mode" --yes > "$LOG_DIR/create_${mode}.log" 2>&1; then
247             generated=$((generated + 1))
248             log "INFO" "Successfully created: $mode"
249         else
250             failed=$((failed + 1))
251             log "ERROR" "Failed to create test image for $mode (check $LOG_DIR/create_${mode}.log)"
252         fi
253     fi
254 done
```

This generation step is critical; it ensures that the corrupted states (like `int_overflow` or `trunc_meta`) are freshly prepared and available to test the kernel's integrity validation logic.

3. QEMU Execution Orchestration (run_single_test)

The `run_single_test` function is the wrapper for launching the virtualized environment using the dedicated, strict launcher `qemu_tests.sh`.

Dual Timeout Strategy

The script manages a crucial **dual-timeout strategy** to optimize testing:

1. **Rejection Tests** (e.g., `meta1`): Use `TEST_TIMEOUT` (default **15s**). If a corrupted image boots for longer than 15s, it is likely a security failure, and the test should terminate quickly.
2. **Clean Boot Tests** (e.g., `best_case`): Use `BOOT_TIMEOUT` (default **45s**). This longer period allows the uncorrupted system sufficient time to complete the full boot process, reach the login prompt, and then be terminated by the timeout command.

```
289 | # Disable exit-on-error for QEMU execution
290 | # timeout returns 124 on timeout, QEMU may return various codes
291 | set +e
292 | timeout -k 5 "${timeout_val}" ./qemu_tests.sh "$test_image_abs" "$serial_log" >> "$log_file" 2>&1
293 | qemu_exit=$?
294 | set -e
```

The script temporarily disables Bash error checking (`set +e`) because a successful security event like a kernel panic or a rejection will result in a non-zero exit code, and a successful boot will result in the timeout utility returning **124**, all of which would halt the script prematurely under strict execution.

The script then performs a robust cleanup:

```
312 | # Force kill any remaining QEMU processes
313 | pkill -9 -f "qemu-system.*$test_name" 2>/dev/null || true
314 | pkill -9 -f "qemu-system.*$(basename "$test_image")" 2>/dev/null || true
315 | sleep 1
```

4. Log Analysis and Behavior Classification (analyze_boot_behavior)

After QEMU exits, the `analyze_boot_behavior` function processes the serial console output to determine the final system state. This classification is the heart of the test results.

The function classifies the outcome into distinct, actionable states based on kernel log messages:

```
341 | # Check for kernel panic first (highest priority)
342 | if grep -q -i "kernel panic" "$log_file"; then
343 |     echo "KERNEL_PANIC"
344 |     return 0
345 | fi
346 |
347 | # Check for successful boot indicators
348 | if grep -q -i "login:" "$log_file"; then
349 |     echo "BOOT_SUCCESS"
350 |     return 0
351 | fi
352 |
353 | if grep -q -i "Welcome to" "$log_file" && grep -q -i "systemd" "$log_file"; then
354 |     echo "BOOT_SUCCESS"
355 |     return 0
356 | fi
```

```

358     # Check for explicit rejection messages
359     if grep -q -i "dm-verity-autoboot: untrusted" "$log_file" ||
360        grep -q -i "signature verification FAILED" "$log_file" ||
361        grep -q -i "unknown tail magic" "$log_file" ||
362        grep -q -i "corrupted metadata" "$log_file" ||
363        grep -q -i "invalid signature" "$log_file" ||
364        grep -q -i "hash verification failed" "$log_file" ||
365        grep -q -i "VALIDATION FAILED" "$log_file" ||
366        grep -q -i "verification failed" "$log_file"; then
367         echo "REJECTED"
368         return 0
369     fi
370

```

The script also distinguishes between different timeout scenarios by checking the QEMU exit code (124) against whether any boot progress (e.g., "Linux version") was logged:

```

371     # Check for timeout (QEMU exit code 124)
372     if grep -q "QEMU exit code: 124" "$log_file"; then
373         # Timeout occurred - check if there was boot progress
374         if grep -q -i "Booting" "$log_file" || grep -q -i "Linux version" "$log_file"; then
375             echo "TIMEOUT_WITH_PROGRESS"
376         else
377             echo "TIMEOUT_NO_OUTPUT"
378         fi
379         return 0
380     fi

```

5. Result Evaluation and Reporting

The `evaluate_test_result` function compares the derived `actual_behavior` against the `expected_behavior` from the `TEST_MODES` array. This step determines the final pass/fail status.

Security Checkpoint

The most critical evaluation is detecting a **Security Violation**, which occurs when a corrupted image is allowed to boot successfully:

```

417         "REJECT"|"PANIC")
418         # For corruption tests: we expect rejection
419         case "$actual_behavior" in
420             "REJECTED"|"KERNEL_PANIC")
421                 return 0 # PASSED - rejected as expected
422                 ;;
423             "TIMEOUT_WITH_PROGRESS"|"TIMEOUT_NO_OUTPUT")
424                 # Timeout can indicate rejection (system halted)
425                 return 0 # PASSED - likely rejected
426                 ;;
427             "BOOT_SUCCESS")
428                 return 1 # FAILED - corrupted image should NOT boot!
429                 ;;
430             *)
431                 return 2 # INCONCLUSIVE
432                 ;;
433         esac
434         ;;
435     *)
436         return 2 # INCONCLUSIVE - unknown expected behavior
437         ;;
438     esac
439 }

```

If an image intended for rejection achieves `BOOT_SUCCESS`, the function returns **1** (FAILED), triggering a red-coded message in the final report: `SECURITY VIOLATION`. All other outcomes (matching `REJECTED`, `KERNEL_PANIC`, or expected `TIMEOUT`) result in a **0** (PASSED).

Final Summary

The main function loops through the results, prints a color-coded status update for each test, and calls `print_summary` to aggregate all results. This final report includes the total counts for passed, failed, and inconclusive tests, providing a clear audit trail of the security validation status. The script exits with status **1** if any security failures were detected.

=====			
TEST SUMMARY			
=====			
TEST	RESULT	BEHAVIOR	EXPECTED

metal	PASSED	KERNEL_PANIC	PANIC
sig1	PASSED	KERNEL_PANIC	PANIC
int_overflow	PASSED	REJECTED	REJECT
buf_overflow	PASSED	REJECTED	REJECT
trunc_meta	PASSED	REJECTED	REJECT
bad_offsets	PASSED	REJECTED	REJECT
sanitize	PASSED	REJECTED	REJECT
best_case	PASSED	BOOT_SUCCESS	BOOT

Total Tests:	8		
Security PASS:	8		
Security FAIL:	0		
Inconclusive:	0		

End of Document